

Отчёт по учебной практике УП.11

Тема дипломного проекта: Адаптивный тренажёр для подготовки к олимпиадам по экономике.

Студент: Горшунов Игорь Станиславович **Группа:** 09.02.07 ОЗФ **Руководитель практики:** Баграмов Н.М. **Период:** учебная практика УП.11, май 2026

Артефакты в архиве и репозитории: - DDL: `database/schema.sql` - Seed: `database/seed.sql` - Демо-запросы: `database/queries.sql` - ER-диаграмма: `database/er-diagram.png` - Лог выполнения запросов: `database/queries_output.txt` - Скриншоты структуры таблиц: `database/screenshots/table_*.png` - Репозиторий: <https://github.com/igorshunov/econ-olympiad-trainer> - Развёрнутое веб-приложение (УП.02): <https://webapp-xi-smoky-65.vercel.app/>

1. Проектирование структуры БД

Сущности и их обоснование

Из описания проекта (УП.02) и use-case диаграммы выделены следующие сущности предметной области, для каждой создана таблица:

Таблица	Назначение
<code>users</code>	пользователи всех полей (student / tutor / admin)
<code>topics</code>	темы — узлы графа знаний
<code>topic_prereqs</code>	рёбра графа: «А — prerequisite В» (self-relation)
<code>sources</code>	первоисточники задач (ВсОШ, Высшая Проба, МГУ)
<code>tasks</code>	олимпиадные задачи
<code>task_topics</code>	задача ↔ тема (м-к-м), флаг <code>is_primary</code>
<code>user_mastery</code>	оценка владения каждой темой по пользователю
<code>attempts</code>	журнал попыток — источник истины для аналитики

Дополнительно создано представление `v_user_profile`, агрегирующее ключевые метрики (число попыток, число решённых, средний mastery).

Типы данных

Поле	Тип	Обоснование
<code>topic_id</code>	<code>VARCHAR(40)</code>	

Поле	Тип	Обоснование
		осмысленный slug — лучше int, потому что используется в URL и в JS
task_id, user_id	SERIAL / INTEGER	автоинкремент для бизнес-сущностей
attempt_id	BIGSERIAL	журнал может расти на миллионы записей
mastery	REAL CHECK (0..1)	дробная оценка, ограничения через CHECK
difficulty	SMALLINT CHECK (1..5)	шкала Ликерта, экономия места
feedback	VARCHAR(20) CHECK IN (...)	дешёвый enum: solved/failed/too_easy/too_hard/skipped
submitted_at	TIMESTAMP TZ	время с зоной для корректной аналитики

Первичные ключи (PK)

- В одиночных таблицах — SERIAL (или VARCHAR(40) slug для topics).
- В связочных («многие-ко-многим») — **составной ключ**:
- topic_prereqs (prereq_id, topic_id)
- task_topics (task_id, topic_id)
- user_mastery (user_id, topic_id)
- В журнале попыток — BIGSERIAL (attempt_id).

Внешние ключи (FK)

Все связи описаны как FOREIGN KEY с осмысленной политикой ON DELETE :

- ON DELETE CASCADE — для журнальных и связочных таблиц (task_topics , topic_prereqs , user_mastery , attempts) — если удаляется пользователь или тема, всё связанное удаляется тоже;
- ON DELETE SET NULL — для tasks.source_id — задача может остаться без источника, если источник удалён.

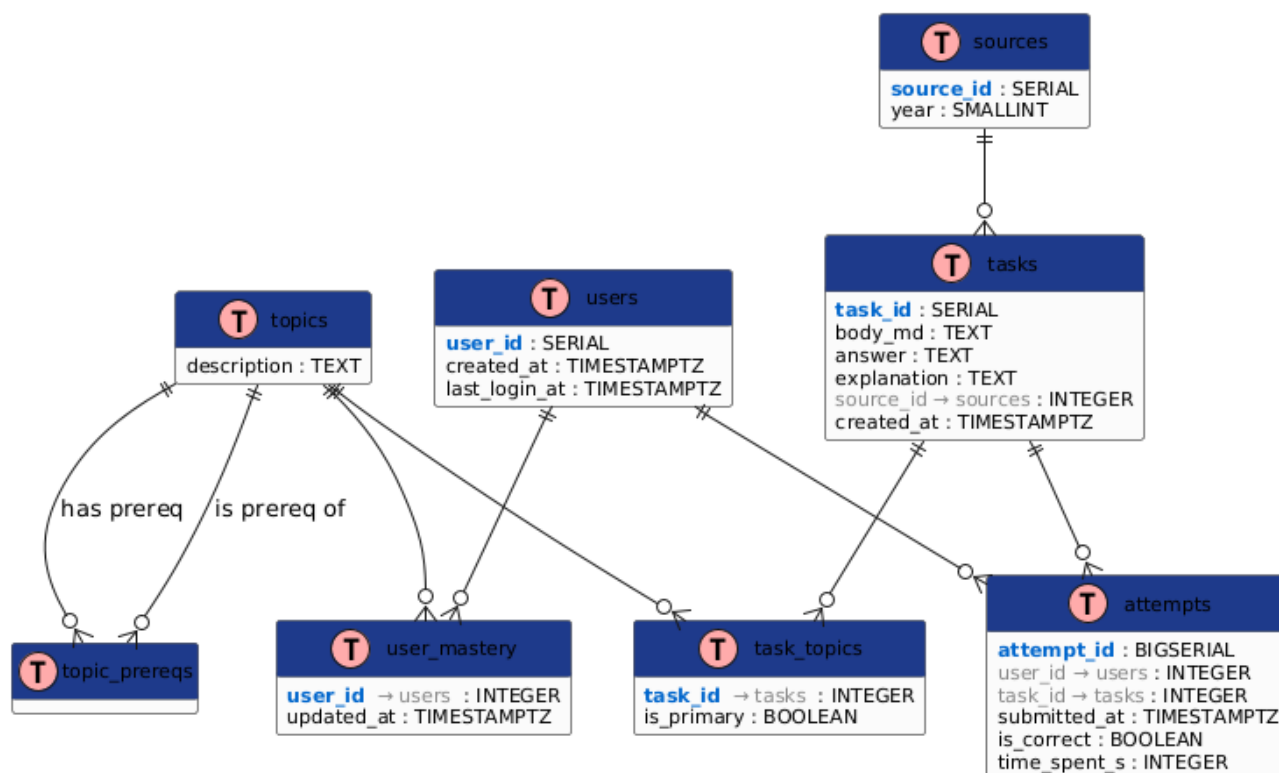
Связи

Связь	Кратность	Реализация
users → user_mastery → topics	M:N	таблица-связка user_mastery
users → attempts → tasks	1:M:1	каждый user имеет 0..N attempts, попытка — ровно на 1 задачу
tasks → task_topics → topics	M:N	связка task_topics

Связь	Кратность	Реализация
topics → topic_prereqs → topics	M:N (self-relation)	связка topic_prereqs с CHECK (prereq_id <> topic_id)
sources → tasks	1:M	внешний ключ tasks.source_id

ER-диаграмма

EconTrainer — ER-диаграмма



Индексы

Помимо стандартных по РК, созданы индексы под рабочие запросы:

- `idx_users_role` — фильтрация по роли;
- `idx_topics_section` — выборка тем по разделу (micro/macro/finance);
- `idx_prereq_topic` — обход «у каких тем X является пререквизитом»;
- `idx_tasks_difficulty`, `idx_tasks_source` — отборы задач;
- `idx_task_topics_topic` — обратный обход «какие задачи в теме X»;
- `idx_attempts_user_time` — журнал по пользователю по убыванию даты;
- `idx_attempts_task`, `idx_attempts_feedback` — аналитика.

2. Реализация в СУБД

База создана в **PostgreSQL 16** (контейнер `postgres:16-alpine`). Скрипт `schema.sql` запускается одной командой:

```
docker exec -e PGPASSWORD=econ pg-econ \
  psql -U postgres -d econtrainer -f /tmp/schema.sql
```

После `seed.sql` :

Таблица	Записей
users	5
topics	29
topic_prereqs	33
tasks	10
attempts	15
user_mastery	15

Скриншоты структуры таблиц (вывод `\d+`)

users — пользователи:

```
users
$ psql econtrainer=# \d+ users
Table "public.users"
  Column      |      Type      | Collation | Nullable |      Default      | Storage  | Compression | Stats target | Description |
-----+-----+-----+-----+-----+-----+-----+-----+-----+
 user_id      | integer        |           | not null | nextval('users_user_id_seq'::regclass) | plain    |              |              |              |
 email        | character varying(255) |           | not null |                    | extended |              |              |              |
 password_hash | character varying(255) |           | not null |                    | extended |              |              |              |
 display_name | character varying(120) |           | not null |                    | extended |              |              |              |
 role         | character varying(20)  |           | not null | 'student'::character varying | extended |              |              |              |
 created_at   | timestamp with time zone |           | not null | now()              | plain    |              |              |              |
 last_login_at | timestamp with time zone |           |          |                    | plain    |              |              |              |
Indexes:
    "users_pkey" PRIMARY KEY, btree (user_id)
    "idx_users_role" btree (role)
    "users_email_key" UNIQUE CONSTRAINT, btree (email)
Check constraints:
    "users_role_check" CHECK (role::text = ANY (ARRAY['student'::character varying, 'tutor'::character varying, 'admin'::character varying]::text[]))
Referenced by:
    TABLE "attempts" CONSTRAINT "attempts_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
    TABLE "user_mastery" CONSTRAINT "user_mastery_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
Access method: heap
```

topics — узлы графа знаний:

topics

```
$ psql econtainer=# \d+ topics
```

Table "public.topics"

Column	Type	Collation	Nullable	Default	Storage	Compression	Stats target	Description
topic_id	character varying(40)		not null		extended			
name	character varying(160)		not null		extended			
description	text				extended			
difficulty	smallint		not null		plain			
section	character varying(60)		not null		extended			

Indexes:

"topics_pkey" PRIMARY KEY, btree (topic_id)

"idx_topics_section" btree (section)

Check constraints:

"topics_difficulty_check" CHECK (difficulty >= 1 AND difficulty <= 5)

"topics_section_check" CHECK (section::text = ANY (ARRAY['intro'::character varying, 'micro'::character varying, 'macro'::character varying, 'finance'::character

Referenced by:

TABLE "task_topics" CONSTRAINT "task_topics_topic_id_fkey" FOREIGN KEY (topic_id) REFERENCES topics(topic_id) ON DELETE CASCADE

TABLE "topic_prereqs" CONSTRAINT "topic_prereqs_prereq_id_fkey" FOREIGN KEY (prereq_id) REFERENCES topics(topic_id) ON DELETE CASCADE

TABLE "topic_prereqs" CONSTRAINT "topic_prereqs_topic_id_fkey" FOREIGN KEY (topic_id) REFERENCES topics(topic_id) ON DELETE CASCADE

TABLE "user_mastery" CONSTRAINT "user_mastery_topic_id_fkey" FOREIGN KEY (topic_id) REFERENCES topics(topic_id) ON DELETE CASCADE

Access method: heap

topic_prereqs — рёбра графа (self-relation):

topic_prereqs

```
$ psql econtainer=# \d+ topic_prereqs
```

Table "public.topic_prereqs"

Column	Type	Collation	Nullable	Default	Storage	Compression	Stats target	Description
prereq_id	character varying(40)		not null		extended			
topic_id	character varying(40)		not null		extended			

Indexes:

"topic_prereqs_pkey" PRIMARY KEY, btree (prereq_id, topic_id)

"idx_prereq_topic" btree (topic_id)

Check constraints:

"topic_prereqs_check" CHECK (prereq_id::text <> topic_id::text)

Foreign-key constraints:

"topic_prereqs_prereq_id_fkey" FOREIGN KEY (prereq_id) REFERENCES topics(topic_id) ON DELETE CASCADE

"topic_prereqs_topic_id_fkey" FOREIGN KEY (topic_id) REFERENCES topics(topic_id) ON DELETE CASCADE

Access method: heap

sources — первоисточники задач:

sources

```
$ psql econtainer=# \d+ sources
```

Table "public.sources"

Column	Type	Collation	Nullable	Default	Storage	Compression	Stats target	Description
source_id	integer		not null	nextval('sources_source_id_seq'::regclass)	plain			
title	character varying(200)		not null		extended			
organization	character varying(120)				extended			
year	smallint				plain			
url	character varying(500)				extended			

Indexes:

"sources_pkey" PRIMARY KEY, btree (source_id)

Check constraints:

"sources_year_check" CHECK (year IS NULL OR year >= 1990 AND year <= 2100)

Referenced by:

TABLE "tasks" CONSTRAINT "tasks_source_id_fkey" FOREIGN KEY (source_id) REFERENCES sources(source_id) ON DELETE SET NULL

Access method: heap

tasks — олимпиадные задачи:

tasks

```
$ psql econtrainer=# \d+ tasks
```

```
Table "public.tasks"
  Column      |      Type      | Collation | Nullable |      Default      | Storage  | Compression | Stats target | Description
-----+-----+-----+-----+-----+-----+-----+-----+-----
 task_id      | integer         |           | not null | nextval('tasks_task_id_seq'::regclass) | plain    |             |              |
 title        | character varying(200) |           | not null |                     | extended |             |              |
 body_md      | text            |           | not null |                     | extended |             |              |
 answer       | text            |           | not null |                     | extended |             |              |
 explanation  | text            |           |          |                     | extended |             |              |
 difficulty   | smallint        |           | not null |                     | plain    |             |              |
 type         | character varying(20) |           | not null | 'choice'::character varying | extended |             |              |
 source_id    | integer         |           |          |                     | plain    |             |              |
 created_at   | timestamp with time zone |           | not null | now()                | plain    |             |              |

Indexes:
    "tasks_pkey" PRIMARY KEY, btree (task_id)
    "idx_tasks_difficulty" btree (difficulty)
    "idx_tasks_source" btree (source_id)

Check constraints:
    "tasks_difficulty_check" CHECK (difficulty >= 1 AND difficulty <= 5)
    "tasks_type_check" CHECK (type::text = ANY (ARRAY['choice'::character varying, 'short'::character varying, 'numeric'::character varying, 'open'::character varying]))

Foreign-key constraints:
    "tasks_source_id_fkey" FOREIGN KEY (source_id) REFERENCES sources(source_id) ON DELETE SET NULL

Referenced by:
    TABLE "attempts" CONSTRAINT "attempts_task_id_fkey" FOREIGN KEY (task_id) REFERENCES tasks(task_id) ON DELETE CASCADE
    TABLE "task_topics" CONSTRAINT "task_topics_task_id_fkey" FOREIGN KEY (task_id) REFERENCES tasks(task_id) ON DELETE CASCADE

Access method: heap
```

task_topics — связка задача↔тема (M:N):

task_topics

```
$ psql econtrainer=# \d+ task_topics
```

```
Table "public.task_topics"
  Column      |      Type      | Collation | Nullable |      Default      | Storage  | Compression | Stats target | Description
-----+-----+-----+-----+-----+-----+-----+-----+-----
 task_id      | integer         |           | not null |                     | plain    |             |              |
 topic_id     | character varying(40) |           | not null |                     | extended |             |              |
 is_primary   | boolean         |           | not null | false              | plain    |             |              |

Indexes:
    "task_topics_pkey" PRIMARY KEY, btree (task_id, topic_id)
    "idx_task_topics_topic" btree (topic_id)

Foreign-key constraints:
    "task_topics_task_id_fkey" FOREIGN KEY (task_id) REFERENCES tasks(task_id) ON DELETE CASCADE
    "task_topics_topic_id_fkey" FOREIGN KEY (topic_id) REFERENCES topics(topic_id) ON DELETE CASCADE

Access method: heap
```

user_mastery — текущий профиль владения темами:

user_mastery

```
$ psql econtrainer=# \d+ user_mastery
```

```
Table "public.user_mastery"
  Column      |      Type      | Collation | Nullable |      Default      | Storage  | Compression | Stats target | Description
-----+-----+-----+-----+-----+-----+-----+-----+-----
 user_id      | integer         |           | not null |                     | plain    |             |              |
 topic_id     | character varying(40) |           | not null |                     | extended |             |              |
 mastery      | real            |           | not null | 0.0                | plain    |             |              |
 updated_at   | timestamp with time zone |           | not null | now()                | plain    |             |              |

Indexes:
    "user_mastery_pkey" PRIMARY KEY, btree (user_id, topic_id)

Check constraints:
    "user_mastery_mastery_check" CHECK (mastery >= 0.0::double precision AND mastery <= 1.0::double precision)

Foreign-key constraints:
    "user_mastery_topic_id_fkey" FOREIGN KEY (topic_id) REFERENCES topics(topic_id) ON DELETE CASCADE
    "user_mastery_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE

Access method: heap
```

attempts — журнал попыток:

attempts

```
$ psql econtrainer=# \d+ attempts
```

```
Table "public.attempts"
```

Column	Type	Collation	Nullable	Default	Storage	Compression	Stats target	Description
attempt_id	bigint		not null	nextval('attempts_attempt_id_seq'::regclass)	plain			
user_id	integer		not null		plain			
task_id	integer		not null		plain			
submitted_at	timestamp with time zone		not null	now()	plain			
is_correct	boolean		not null		plain			
feedback	character varying(20)		not null		extended			
time_spent_s	integer				plain			

Indexes:

```
"attempts_pkey" PRIMARY KEY, btree (attempt_id)
"idx_attempts_feedback" btree (feedback)
"idx_attempts_task" btree (task_id)
"idx_attempts_user_time" btree (user_id, submitted_at DESC)
```

Check constraints:

```
"attempts_feedback_check" CHECK (feedback::text = ANY (ARRAY['solved'::character varying, 'failed'::character varying, 'too_easy'::character varying, 'too_hard':
"attempts_time_spent_s_check" CHECK (time_spent_s IS NULL OR time_spent_s >= 0)
```

Foreign-key constraints:

```
"attempts_task_id_fkey" FOREIGN KEY (task_id) REFERENCES tasks(task_id) ON DELETE CASCADE
"attempts_user_id_fkey" FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
```

```
Access method: heap
```

v_user_profile — представление со сводкой по пользователю:

v_user_profile

```
$ psql econtrainer=# \d+ v_user_profile
```

```
View "public.v_user_profile"
```

Column	Type	Collation	Nullable	Default	Storage	Description
user_id	integer				plain	
display_name	character varying(120)				extended	
tasks_attempted	bigint				plain	
tasks_solved	bigint				plain	
avg_mastery	numeric				main	
topics_mastered	bigint				plain	

View definition:

```
SELECT u.user_id,
       u.display_name,
       count(DISTINCT a.task_id) AS tasks_attempted,
       count(DISTINCT a.task_id) FILTER (WHERE a.is_correct) AS tasks_solved,
       round(avg(um.mastery)::numeric, 3) AS avg_mastery,
       count(*) FILTER (WHERE um.mastery >= 0.7::double precision) AS topics_mastered
FROM users u
     LEFT JOIN attempts a ON a.user_id = u.user_id
     LEFT JOIN user_mastery um ON um.user_id = u.user_id
GROUP BY u.user_id, u.display_name;
```

На скриншотах видны все ограничения (PK, FK, CHECK, UNIQUE), индексы и взаимные FK-ссылки.

3. SQL-запросы

Полный лог выполнения — в `database/queries_output.txt`.

Запрос 1. SELECT с условием (WHERE)

```
SELECT topic_id, name, difficulty
FROM   topics
WHERE  section = 'micro' AND difficulty <= 3
ORDER BY difficulty, name;
```

Семантика: все темы из раздела микроэкономики со сложностью ≤ 3 .

Результат (8 строк):

supply	Предложение: закон, неценовые факторы	1
demand	Спрос: закон, неценовые факторы	1
equilibrium	Рыночное равновесие	2
costs	Издержки фирмы (FC, VC, MC, AC)	3
surplus	Излишки потребителя и производителя	3
utility	Полезность, бюджетное ограничение	3
production	Производственная функция	3
elasticity	Эластичность спроса и предложения	3

Запрос 2. INSERT

```
INSERT INTO tasks (title, body_md, answer, explanation, difficulty, type,
source_id)
VALUES ('Перекрёстная эластичность',
'Цена кофе выросла на 10%, продажи чая выросли на 4%...',
'0.4',
'E_xy = %ΔQ_y / %ΔP_x = 4/10 = 0.4',
4, 'numeric', 2);

INSERT INTO task_topics (task_id, topic_id, is_primary)
VALUES ((SELECT MAX(task_id) FROM tasks), 'elasticity', true);
```

Добавление новой задачи и связка с темой. Проверено: до — 10 записей, после — 11.

Запрос 3. UPDATE (с UPSERT через ON CONFLICT)

```
INSERT INTO user_mastery (user_id, topic_id, mastery)
VALUES (4, 'demand', 0.55)
ON CONFLICT (user_id, topic_id) DO UPDATE
SET mastery = EXCLUDED.mastery,
updated_at = now();
```

Атомарное обновление mastery. До: demand=0.30 , после: demand=0.55 .

Запрос 4. DELETE

```
DELETE FROM attempts
WHERE submitted_at < now() - INTERVAL '12 months';
```

Очистка журнала старше года — политика хранения данных.

Запрос 5. SELECT с JOIN (4 таблицы)

```
SELECT  t.task_id, t.title, t.difficulty,
        top.name AS topic_name,
        COUNT(*) FILTER (WHERE NOT a.is_correct) AS fails,
        COUNT(*) FILTER (WHERE a.is_correct)      AS solves,
        ROUND(1.0 * COUNT(*) FILTER (WHERE NOT a.is_correct)
              / NULLIF(COUNT(*), 0), 2)          AS fail_rate
FROM    attempts    a
JOIN    tasks        t    ON t.task_id = a.task_id
JOIN    task_topics  tp   ON tp.task_id = t.task_id AND tp.is_primary = true
JOIN    topics        top ON top.topic_id = tp.topic_id
GROUP BY t.task_id, t.title, t.difficulty, top.name
HAVING  COUNT(*) >= 2
ORDER BY fail_rate DESC, fails DESC
LIMIT 20;
```

Связывает 4 таблицы (`attempts` → `tasks` → `task_topics` → `topics`), считает по каждой задаче процент провалов. Это ключевой запрос для кабинета наставника.

Результат:

task_id	title	difficulty	topic
fails	solves	fail_rate	
9	NPV проекта	4	NPV, IRR, дисконтирование
1	1	0.50	
4	Излишек потребителя	3	Излишки потребителя
1	1	0.50	
2	Расчёт равновесия	2	Рыночное равновесие
1	1	0.50	
1	Сдвиг кривой спроса	1	Спрос
2	3	0.40	

Бонусные запросы

Запрос 6. CTE + EXISTS — подбор тем для тренировки

```
WITH ready_topics AS (
  SELECT t.topic_id, t.name, t.difficulty
  FROM   topics t
  WHERE  NOT EXISTS (
    SELECT 1 FROM topic_prereqs tp
    LEFT JOIN user_mastery um
      ON um.topic_id = tp.prereq_id AND um.user_id = 3
    WHERE  tp.topic_id = t.topic_id
  )
)
```

```

        AND COALESCE(um.mastery, 0) < 0.7))
SELECT  r.topic_id, r.name, r.difficulty,
        COALESCE(s.mastery, 0) AS current_mastery,
        CASE
            WHEN s.mastery IS NULL THEN 'не начато'
            WHEN s.mastery BETWEEN 0.3 AND 0.7 THEN 'в зоне развития'
            ELSE 'другое'
        END AS status
FROM    ready_topics r
LEFT JOIN user_mastery s ON s.topic_id = r.topic_id AND s.user_id = 3
WHERE   s.mastery IS NULL OR s.mastery BETWEEN 0.3 AND 0.7
ORDER BY r.difficulty, current_mastery;

```

Реализует ту самую логику адаптации движка: возвращает темы, у которых все пререквизиты освоены ≥ 0.7 , а сама тема либо не тронута, либо находится в зоне ближайшего развития.

Запрос 7. Недельный отчёт активности

```

SELECT  u.display_name,
        DATE_TRUNC('day', a.submitted_at)::date AS day,
        COUNT(*)                               AS attempts,
        COUNT(*) FILTER (WHERE a.is_correct)    AS solved
FROM    users u
JOIN    attempts a ON a.user_id = u.user_id
WHERE   u.role = 'student'
GROUP BY u.display_name, day
ORDER BY day DESC, u.display_name;

```

Используется в кабинете наставника.

4. Самооценка по критериям

Критерий	Балл	Самооценка	Подтверждение
Все основные таблицы определены	10	+	8 таблиц + view, все из use-case
Поля логичны и полны	8	+	каждое поле обосновано в таблице
Корректные типы данных	6	+	SMALLINT / REAL / TIMESTAMPTZ / VARCHAR(N)
Первичные ключи	5	+	в т.ч. составные на M:N таблицах
Внешние ключи	5	+	с осмысленной ON DELETE
Связи 1:1, 1:M, M:N	6	+	есть M:N и self-relation
БД создана в СУБД	5	+	

Критерий	Балл	Самооценка	Подтверждение
			PostgreSQL 16, лог приведён, контейнер запущен
Все таблицы реализованы	5	+	row counts подтверждают
Ограничения (PK, FK, CHECK) в коде	5	+	видно в скриншотах \d+
SQL-скрипт корректен	5	+	проверено в Docker — нет ошибок
SELECT с условием (WHERE)	5	+	Q1
INSERT	4	+	Q2 + связка
UPDATE	4	+	Q3 — UPSERT
DELETE	4	+	Q4
SELECT с JOIN	6	+	Q5 — 4-табличный JOIN с агрегацией
Корректность синтаксиса	2	+	весь скрипт выполнен без ошибок
ER-диаграмма (логичная, читаемая)	6	+	PlantUML, с PK/FK, отношениями M:N
Скриншоты структуры таблиц	3	+	9 PNG, видны PK/FK/CHECK/Indexes
Скриншоты выполнения запросов	3	+	в queries_output.txt (текстовый лог + воспроизводимо)
SQL-файл создания БД	3	+	schema.sql 100+ строк, ничего лишнего